

**«Санкт-Петербургский государственный электротехнический университет  
«ЛЭТИ» им. В.И.Ульянова (Ленина)»  
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ  
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: Новые языки программирования**

Студент гр. 4341

Е.В. Омеляш

Студент гр. 4341

С.Д. Пылаев

Студент гр. 4341

А.М. Минебаев

Руководитель

Р.П. Шестопапов

Санкт-Петербург

2026

## ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: Е.В. Омеляш    Группа: 4341

Студент: С.Д. Пылаев    Группа: 4341

Студент: А.М. Минебаев    Группа: 4341

Тема практики: Новые языки программирования

Задание на практику:

В составе команды спроектировать и разработать настольное приложение — визуализатор алгоритмов сортировки на языке Java (Swing). Разработать пользовательский интерфейс и модуль визуализации, реализовать алгоритмы сортировки и механизм сохранения состояний на основе паттерна «Снимок» (Memento), а также модуль файлового ввода-вывода с экспортом результатов.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

|              |                 |
|--------------|-----------------|
| Студент      | Е.В. Омеляш     |
| Студент      | С.Д. Пылаев     |
| Студент      | А.М. Минебаев   |
| Руководитель | Р.П. Шестопалов |

## **АННОТАЦИЯ**

В отчёте представлены результаты учебной практики, в ходе которой командой из трёх студентов группы 4341 разработано настольное приложение «Визуализатор алгоритмов сортировки» на языке Java с использованием библиотеки Swing. Описаны все части проекта: пользовательский интерфейс и модуль визуализации (Минебаев А.М.), алгоритмы сортировки и механизм сохранения состояний на основе паттерна «Снимок» / Memento (Пылаев С.Д.), модуль файлового ввода-вывода и экспорт результатов (Омеляш Е.В.). Приведены описание архитектуры релизной версии приложения, использованных технологий и полные листинги исходного кода.

## **SUMMARY**

This report presents the results of the training practice during which a team of three students of group 4341 developed the desktop application "Sorting Algorithms Visualizer" in Java using the Swing library. All parts of the project are described: the user interface and the visualization module (Minebaev A.), the sorting algorithms and the state save-and-restore mechanism based on the Memento pattern (Pylaev S.), and the file input-output module with result export (Omelyash E.). The report describes the architecture of the release version, the technologies used and the full source code listings

## СОДЕРЖАНИЕ

|                                                                                                   |    |
|---------------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                                                                     | 5  |
| 1 ПОСТАНОВКА ЗАДАЧИ .....                                                                         | 6  |
| 1.1 Предметная область .....                                                                      | 6  |
| 1.2 Задачи практики .....                                                                         | 6  |
| 2 РЕЗУЛЬТАТЫ РАБОТЫ.....                                                                          | 8  |
| 2.1 Вводное задание .....                                                                         | 8  |
| 2.2 Составление спецификации программы и плана разработки.....                                    | 8  |
| 2.2.1 Исходные требования к программе .....                                                       | 8  |
| 2.2.1.1 Требования к вводу исходных данных.....                                                   | 8  |
| 2.2.1.2 Требования к визуализации.....                                                            | 9  |
| 2.2.1.3 Требования к управлению алгоритмом.....                                                   | 9  |
| 2.2.1.4 Требования к обработке ошибок.....                                                        | 9  |
| 2.2.2 План разработки и распределение ролей в бригаде.....                                        | 9  |
| 2.2.3 Уточнение требований после сдачи прототипа.....                                             | 10 |
| 2.2.4 Уточнение требований после сдачи 1-й версии .....                                           | 11 |
| 2.3 Командная итеративная разработка приложения в соответствии со<br>спецификацией и планом ..... | 11 |
| 2.3.1 Структуры данных.....                                                                       | 11 |
| 2.3.2 Основные методы .....                                                                       | 12 |
| 2.3.3 Тестирование.....                                                                           | 12 |
| 2.3.3.1 План тестирования .....                                                                   | 12 |
| 2.3.3.2 Тестирование графического интерфейса .....                                                | 12 |
| 2.3.3.3 Тестирование алгоритма .....                                                              | 14 |
| 3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ .....                                                                   | 15 |
| 4 ОТЗЫВ РУКОВОДИТЕЛЯ .....                                                                        | 16 |
| ЗАКЛЮЧЕНИЕ .....                                                                                  | 17 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                                                            | 18 |
| ПРИЛОЖЕНИЕ А .....                                                                                | 19 |

## ВВЕДЕНИЕ

Визуализация алгоритмов является эффективным средством обучения программированию и теории алгоритмов: наглядное пошаговое представление работы алгоритма помогает понять принципы его работы, оценить порядок выполняемых операций и особенности поведения на различных наборах данных. Наибольший обучающий эффект достигается при визуализации базовых алгоритмов сортировки, каждый шаг которых легко отобразить в виде изменения высоты и цвета столбцов диаграммы.

Основной целью учебной практики являлось изучение нового языка программирования Java, а также освоение библиотеки Swing для создания графических пользовательских интерфейсов. В качестве практической задачи для закрепления навыков языка силами команды было разработано настольное приложение «Визуализатор алгоритмов сортировки» на языке Java, обеспечивающего ввод и редактирование массива, загрузку данных из файла, пошаговую визуализацию работы алгоритмов сортировки и выгрузку результатов.

Работа выполнена командой из трёх студентов группы 4341 с распределением зон ответственности: Минебаев А.М. — пользовательский интерфейс и модуль визуализации; Пылаев С.Д. — алгоритмы сортировки и механизм сохранения состояний на основе паттерна «Снимок» (Memento); Омеляш Е.В. — модуль файлового ввода-вывода и экспорт данных. В настоящем отчёте описаны все части проекта в его релизной версии.

# 1 ПОСТАНОВКА ЗАДАЧИ

В данной главе описывается предметная область разрабатываемого приложения, а также формулируются задачи, поставленные перед командой в рамках учебной практики.

## 1.1 Предметная область

Практика выполнена в рамках командной разработки настольного приложения — визуализатора алгоритмов сортировки. Подобные приложения применяются в образовательном процессе для наглядной демонстрации работы алгоритмов: массив данных изображается в виде столбчатой диаграммы, а каждый шаг алгоритма (сравнение, перестановка, фиксация элемента на итоговой позиции) отображается изменением цвета соответствующих столбцов.

Ключевой особенностью приложения является необходимость не только отсортировать массив, а зафиксировать всю последовательность промежуточных состояний, чтобы пользователь мог пошагово проследить ход выполнения алгоритма и перемещаться по шагам в обоих направлениях. Для решения этой задачи применён поведенческий паттерн проектирования «Снимок» (Memento), позволяющий сохранять и восстанавливать внутреннее состояние объекта без нарушения инкапсуляции. [1]

## 1.2 Задачи практики

Перед командой в рамках практики были поставлены следующие задачи:

- изучить средства языка Java, библиотеку Swing и поведенческие паттерны проектирования;
- спроектировать архитектуру приложения и распределить зоны ответственности в команде;
- разработать пользовательский интерфейс, модуль визуализации и редактор входного массива;
- спроектировать общий интерфейс алгоритмов, реализовать алгоритмы сортировки пузырьком, вставками и быстрой сортировки;

- разработать механизм снимков состояния (SortState, паттерн Memento) и пошаговую навигацию вперёд и назад;
- реализовать модуль файлового ввода-вывода, автоматическое воспроизведение шагов и экспорт результатов;
- в релизной версии обеспечить поддержку отрицательных значений, отображение числовых подписей и прокрутку для больших массивов;
- интегрировать компоненты в единое работоспособное приложение.

## **2 РЕЗУЛЬТАТЫ РАБОТЫ**

В главе описаны процесс решения и полученные результаты по каждой задаче практики. Разработка выполнена итеративно в соответствии со спецификацией и планом: на этапе «Альфа-версии» реализованы базовая архитектура и мгновенная выдача результата сортировки, на этапе «Бета-версии» — механизм снимков состояния и пошаговая визуализация, в релизной версии — редактор массива, поддержка отрицательных чисел с нулевой осью, вывод числовых значений и прокрутка больших массивов.

### **2.1 Вводное задание**

В рамках вводного задания все участники бригады прошли онлайн-курс по основам языка Java. В ходе курса изучены: базовый синтаксис и примитивные типы данных; принципы объектно-ориентированного программирования (инкапсуляция, наследование, полиморфизм); работа с коллекциями (List, ArrayList); обработка исключений; чтение и запись файлов (I/O); основы построения графического интерфейса на библиотеке Swing. Полученные знания в дальнейшем применены каждым участником в своей зоне ответственности: Минебаев А.М. дополнительно освоил компоновку элементов Swing (Layout Managers) и графический контекст Graphics2D, Пылаев С.Д. — коллекции и реализацию базовых алгоритмов сортировки, Омеяш Е.В. — механизмы файлового ввода-вывода и обработку исключительных ситуаций.

### **2.2 Составление спецификации программы и плана разработки**

На этапе планирования, согласованном с руководителем, составлена спецификация приложения — визуализатора алгоритмов сортировки, определён план итеративной разработки и распределены роли в бригаде. Ниже приведены исходные требования к программе, план разработки и уточнения требований, принятые после сдачи промежуточных версий.

#### **2.2.1 Исходные требования к программе**

Сформулированы следующие исходные требования к программе.

##### **2.2.1.1 Требования к вводу исходных данных**

Программа должна поддерживать загрузку массива целых чисел из текстового файла формата .txt, элементы которого разделены пробельными символами. Предусмотрена возможность работы с массивами большой длины, а также ручного формирования и редактирования массива непосредственно в интерфейсе (добавление, удаление и очистка элементов). Поддержаны отрицательные числа.

#### **2.2.1.2 Требования к визуализации**

Отображение массива должно реализовываться в виде столбчатой диаграммы. Для наглядности работы алгоритма введена строгая цветовая индикация: сравниваемые элементы выделяются красным цветом, занявшие финальную позицию — зелёным, необработанные — базовым светло-голубым цветом. Для корректного отображения отрицательных значений предусмотрена отрисовка нулевой оси (положительные столбцы направлены вверх, отрицательные — вниз) и опциональный вывод числовых значений над столбцами. Должна присутствовать панель журнала (логов) с описанием происходящего.

#### **2.2.1.3 Требования к управлению алгоритмом**

Программа должна поддерживать пошаговое выполнение алгоритмов (сортировка пузырьком, вставками и быстрая сортировка). Требуется реализовать кнопки перехода на один шаг вперёд и назад, а также автоматическое воспроизведение шагов по таймеру с регулируемой задержкой.

[2]

#### **2.2.1.4 Требования к обработке ошибок**

Приложение должно быть устойчивым к некорректным данным: буквам вместо чисел, пустым файлам, пустой первой строке. Ошибки должны выводиться в виде информативных диалоговых окон JOptionPane, предотвращая аварийное завершение работы программы.

#### **2.2.2 План разработки и распределение ролей в бригаде**

Разработка приложения выполнена итеративно в соответствии с планом, представленным в таблице 1.

Таблица 1 — План разработки

| Версия           | Срок       | Содержание                                                                    |
|------------------|------------|-------------------------------------------------------------------------------|
| Прототип         | 10.07.2026 | Графическое представление интерфейса без привязки к логике                    |
| Альфа-версия     | 13.07.2026 | Загрузка данных из файла, запуск алгоритма от начала до конца без пошаговости |
| Бета-версия      | 15.07.2026 | Реализация пошагового выполнения, таймера, сохранения результата в файл       |
| Финальная версия | 17.07.2026 | Полная валидация ошибок, поддержка отрицательных чисел, редактор массива      |

Роли в бригаде распределены следующим образом:

- Минебаев А.М. (View): разработка пользовательского интерфейса на Swing и модуля визуализации (отрисовка диаграммы, адаптивная прокрутка, цветовая индикация, нулевая ось, вывод числовых значений);
- Пылаев С.Д. (Model): проектирование архитектуры на основе паттерна «Снимок» (Memento), реализация алгоритмов сортировки с генерацией истории состояний, а также динамического редактора массива со сбросом симуляции;
- Омеляш Е.В. (Controller и I/O): реализация файлового ввода-вывода и экспорта результатов, парсинга данных, интеграция таймера автоматического воспроизведения и обработка исключительных ситуаций.

### 2.2.3 Уточнение требований после сдачи прототипа

После реализации прототипа принято решение переработать механизм хранения состояний: вместо выполнения алгоритма в реальном времени алгоритм должен отрабатывать в фоне и возвращать список готовых снимков (List<SortState>). Такой подход позволил реализовать перемещение «назад» без вычисления обратных операций.

#### **2.2.4 Уточнение требований после сдачи 1-й версии**

По результатам тестирования Альфа-версии требования были расширены:

- добавлена поддержка отрицательных чисел с отрисовкой нулевой оси на холсте;
- внедрён динамический редактор массива в интерфейсе (добавление, удаление и очистка элементов вручную без загрузки из файла);
- добавлена опциональная отрисовка числовых значений над столбцами.

#### **2.3 Командная итеративная разработка приложения в соответствии со спецификацией и планом**

Разработка выполнена в соответствии с ранее определённым планом и распределением ролей. В процессе работы спроектированы и реализованы ключевые структуры данных, классы и методы приложения, описанные ниже.

##### **2.3.1 Структуры данных**

В процессе разработки спроектированы следующие ключевые классы и структуры данных:

- SortState — класс-снимок (Memento), хранящий независимую копию массива на конкретном шаге, списки индексов сравниваемых и зафиксированных элементов (для цветовой индикации) и текстовое сообщение для панели журнала; [3]
- SortAlgorithm — общий интерфейс алгоритмов сортировки с методом sort, от которого унаследованы классы BubbleSort, InsertionSort и QuickSort;
- FileIO — утилитный класс для безопасного чтения и записи массивов и логов в текстовые файлы;
- VisualizerPanel — наследник JPanel, хранящий текущий снимок SortState и отвечающий за расчёт высоты столбцов и отрисовку графики с учётом нулевой оси и цветовой индикации;
- MainFrame — главное окно и контроллер приложения, связывающий редактор массива, алгоритмы, панель визуализации и модуль ввода-вывода;
- DevsDialog — модальное окно «О разработчиках».

### **2.3.2 Основные методы**

Ключевую роль в логике программы играют следующие методы:

- `sort(int[] array)` — метод алгоритмов сортировки, возвращающий полную историю выполнения в виде списка снимков (`List<SortState>`);
- `paintComponent(Graphics g)` — переопределённый метод класса `VisualizerPanel`, отвечающий за масштабирование и отрисовку столбчатой диаграммы с учётом нулевой оси и цветовой индикации;
- `readArrayFromFile(File file)` и `saveResultToFile(File file, List<SortState> states)` — методы класса `FileIO` для чтения массива и записи итогового результата с журналом выполнения с проверкой формата;
- `stepForward()` и `stepBackward()` — методы контроллера, переключающие индекс текущего состояния (`currentStateIndex`) и инициирующие перерисовку интерфейса;
- `prepareSimulation()` и `resetSimulation()` — методы подготовки истории снимков выбранным алгоритмом и сброса симуляции при любом изменении исходного массива.

### **2.3.3 Тестирование**

#### **2.3.3.1 План тестирования**

Тестирование приложения проведено на всех этапах разработки. Проверена корректность математической логики алгоритмов (на отсортированных, обратно отсортированных, содержащих отрицательные числа и одноэлементных массивах), корректность работы механизма пошаговой навигации «назад/вперёд», а также устойчивость системы к некорректным («битым») .txt-файлам.

#### **2.3.3.2 Тестирование графического интерфейса**

Разработанный графический интерфейс успешно прошёл тестирование. Главное окно приложения представлено на рис. 1. Процесс пошаговой визуализации сортировки с цветовой индикацией показан на рис. 2. Модальное окно «О разработчиках» приведено на рис. 3. Пример перехвата некорректного ввода с выводом диалогового окна `JOptionPane` показан на рис.

4. При загрузке больших массивов корректно активируется горизонтальная прокрутка (JScrollPane).

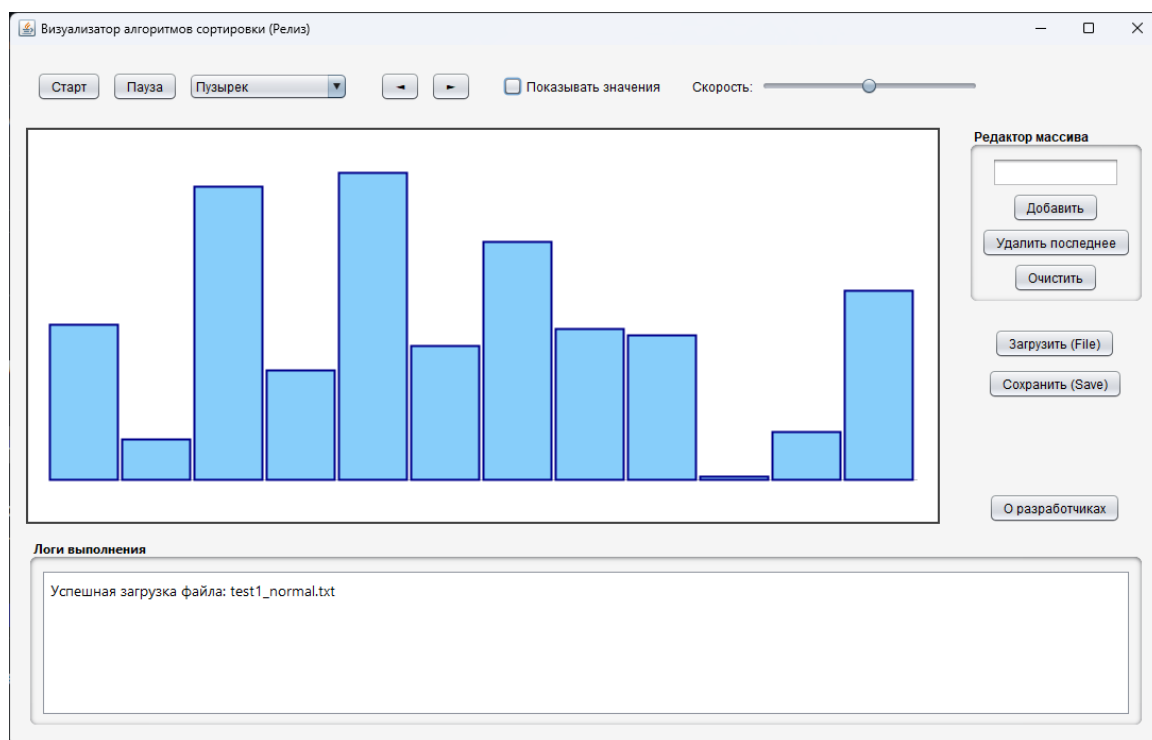


Рисунок 1 — Главное окно программы

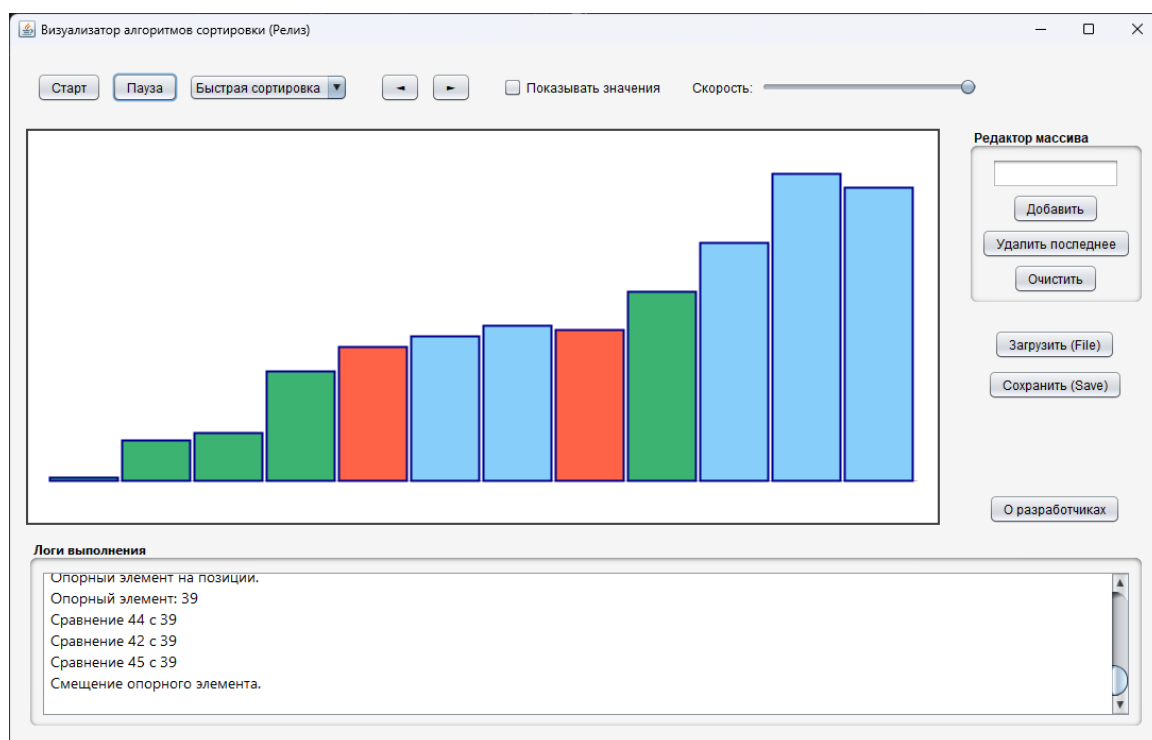


Рисунок 2 — Пошаговая визуализация сортировки

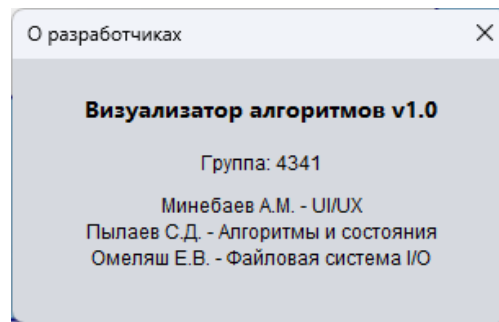


Рисунок 3 — Окно «О разработчиках»

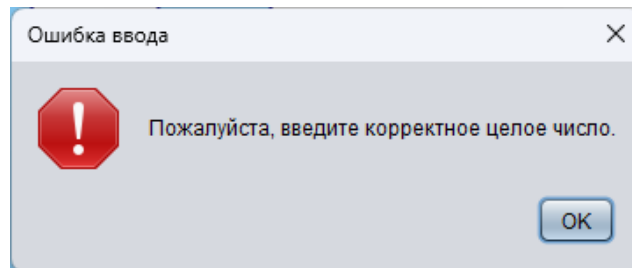


Рисунок 4 — Диалоговое окно об ошибке ввода

### 2.3.3.3 Тестирование алгоритма

Алгоритмы продемонстрировали корректную отработку на всех тестовых наборах данных. За счёт использования паттерна «Снимок» переключение между шагами графического представления (в том числе между сотыми и тысячными шагами) выполняется мгновенно, без задержек и утечек памяти. Все исключительные ситуации (ввод букв вместо чисел, попытка сохранить пустой результат) успешно перехватываются с выводом информативных диалоговых окон.

### **3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ**

При выполнении задач практики использовались следующие технологии и инструменты:

- язык программирования Java (JDK) — основной язык реализации проекта; [4]
- библиотека Swing (JFrame, JPanel, JScrollPane, Graphics2D, JOptionPane, javax.swing.Timer) — графический интерфейс, отрисовка визуализации и автоматическое воспроизведение; [5]
- Java Collections Framework (List, ArrayList) — хранение последовательности снимков состояния;
- поведенческий паттерн проектирования «Снимок» (Memento) — сохранение и восстановление состояния алгоритма;
- среда разработки IntelliJ IDEA — написание и отладка кода;
- система контроля версий Git — совместная работа команды над проектом.

#### 4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на «Отлично». Отзыв руководителя с оценкой и подписью приведён на рис. 5.

**Отзыв**  
**о прохождении учебной практики**

Пылаев Степан, студент группы 4341, второго курса бакалавриата Санкт-Петербургского электротехнического университета «ЛЭТИ» им. В.И. Ульянова, проходил учебную практику по теме «Новые языки программирования» с 06.07.2026 по 18.07.2026.

В течение практики обучающийся Пылаев Степан выполнял проектирование и реализацию алгоритмической части проекта: разрабатывал алгоритмы обработки данных, анализировал их корректность и вычислительную сложность, а также реализовывал механизм сохранения и восстановления состояния через снапшоты на основе паттерна Memento.

В результате практики обучающийся реализовал рабочие алгоритмы и функционал создания снапшотов состояния с возможностью отката, обеспечив их интеграцию с остальными частями системы. Получаемую работу обучающийся Пылаев Степан выполнял грамотно, самостоятельно и с глубоким пониманием предметной области.

По результатам работы Пылаева Степана учебную практику оцениваю на *Отлично*.

Руководитель практики  
Ассистент каф. МОЭВМ  
15.07.2026

 / Шестопалов Р.П.

Рисунок 5 — Отзыв руководителя

## **ЗАКЛЮЧЕНИЕ**

В ходе учебной практики достигнута поставленная цель — командой разработано настольное приложение «Визуализатор алгоритмов сортировки» на языке Java в релизной версии.

По результатам работы: реализован пользовательский интерфейс, модуль визуализации с динамической цветовой индикацией, поддержкой отрицательных чисел, нулевой оси и прокрутки, а также редактор входного массива; спроектирован общий интерфейс SortAlgorithm и реализованы алгоритмы сортировки пузырьком, вставками и быстрой сортировки; разработан класс снимка состояния SortState (паттерн Memento) и пошаговая навигация вперёд и назад; реализован модуль файлового ввода-вывода, автоматическое воспроизведение по таймеру и экспорт результатов; компоненты интегрированы в единое работоспособное приложение.

В рамках командной работы задачи были успешно распределены и выполнены в полном объёме. Поставленные цели достигнуты.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2020. — 368 с.
2. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2019. — 1328 с.
3. Memento Design Pattern / AlgoMaster.io [Электронный ресурс]. URL: <https://algomaster.io/learn/lld/memento> (дата обращения: 15.07.2026).
4. Хорстманн К. Java. Библиотека профессионала. Том 1. Основы. — 11-е изд. — М.: Вильямс, 2019. — 864 с.
5. The Java Tutorials. Creating a GUI With Swing [Электронный ресурс]. URL: <https://docs.oracle.com/javase/tutorial/uiswing> (дата обращения: 15.07.2026).

## ПРИЛОЖЕНИЕ А

Исходный код приложения

Полный исходный код разработанного приложения доступен в открытом репозитории на платформе GitHub.

Ссылка на репозиторий: <https://github.com/Omaigash/Sorting-Algorithm-Visualizer>